

Entry: 1

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem of this approach is that the global lock will protect the acquisition of all the locks. ./vector-avoid-hold-and-wait.c

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) will protect the acquisition of all the other locks.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15

Score 2:

15

Score 3:

15

Entry: 2

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The global locks that protect other locks from acquisition is the main problem because it keeps holding the lock although it is not needed.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for a long time.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

15

Score 3:

15

Entry: 3

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem with this approach is that it can lead to a deadlock if all threads do not acquire the locks in a consistent order.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

5

Score 2:

7

Score 3:

5

Entry: 4

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem with this approach is that the global lock should be unlocked in the end of the code instead of right after the first lock.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for the entire duration of the program.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

10

Score 3:

10

Entry: 5

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem is only one thread could get the mutex of vectors, even if other threads are requiring different vectors.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

12

Score 2:

8

Score 3:

9

Entry: 6

Item: 3

Question:

Now let's look at `vector-avoid-hold-and-wait.c`. What is the main problem with this approach? How does its performance compare to `vector-global-order.c`?

Answer:

Compare to `vector-global-order.c`: The global order approach also avoids deadlocks but allows for more concurrency because it does not hold the lock for the entire duration of the operation.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for the entire duration of the operation, which prevents other threads from acquiring any locks.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15

Score 2:

15

Score 3:

15

Entry: 7

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other approaches?

Answer:

This approach is too rough. Even if the vectors are being manipulated by different threads, the global lock will be under contention.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other vectors) is held by one thread at a time.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15.0

Score 2:

15

Score 3:

15

Entry: 8

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other approaches?

Answer:

After Thread 1 lock the global, if Thread 2 want to run the program and do not use the vector used by Thread 1. Theoretically, it should be fine.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15

Score 2:

15

Score 3:

15

Entry: 9

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other approaches?

Answer:

The thread that is waiting to acquire a lock may be blocked if another thread holds the lock for a long time. The performance of this approach is poor.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for a long time.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

12

Score 2:

10

Score 3:

10

Entry: 10

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem is that the approach requires the threads to acquire the global lock before acquiring the vector, which is inefficient.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other vectors) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

5

Score 2:

5

Score 3:

5

Entry: 11

Item: 3

Question:

What is the main problem with the approach in vector-avoid-hold-and-wait.c? How does its performance compare to vector-

Answer:

The main problem is the global lock causing contention. Without the -p flag, performance is similar to vector-global-order.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain “main problem” (5 points)
- For explain the setting “without the -p flag” (5 points)
- For explain the setting “with the -p flag” (5 points)

Full points:

15

Score 1:

15.0

Score 2:

15

Score 3:

15

Entry: 12

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare with vector-glob

Answer:

The problem is it may lead to deadlock. When running with -p, the performance may be better compared with vector-glob

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

12

Score 3:

10

Entry: 13

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

When running with the parallelism flag (-p), the performance of `vector-avoid-hold-and-wait.c` may be affected by the potential for race conditions.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for the entire duration of the vector-avoid-hold-and-wait.c program.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

7

Score 2:

5

Score 3:

3

Entry: 14

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The global lock will under have conflict when the vectors being manipulated by each thread is different. The run times with the global lock are much longer than the other two approaches.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other vectors) is held for the entire duration of the program.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15

Score 2:

15

Score 3:

15

Entry: 15

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the version in vector-avoid-hold-and-wait.c?

Answer:

The problem is that conflict will be occurred when the vectors in each thread are not equal. Compared to the version in vector-avoid-hold-and-wait.c, the performance is worse.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other vectors) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

5

Score 3:

7

Entry: 16

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to vector-try-and-wait.c?

Answer:

There is no need for the approach to get both two values' lock. The performance is faster than vector-try-and-wait but slower than vector-avoid-hold-and-wait.c.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10

Score 2:

5

Score 3:

8

Entry: 17

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

Without -p With -p ./vector-try-wait -d -n 2 -l 100000 -t ./vector-try-wait -d -n 2 -l 100000 -t -p Retries: 516458 Retries: 0 Times: 100000 Times: 100000

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

7

Score 2:

5

Score 3:

5

Entry: 18

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two methods?

Answer:

With -p it is slightly slower than global order. Without -p it is in the middle between two methods.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

5.0

Score 2:

5

Score 3:

4

Entry: 19

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The mutex lock cannot work properly as there are three mutex lock however, the order of releasing the lock is not same. The global lock is not sufficient to protect the acquisition of all the other locks.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

8

Score 2:

9

Score 3:

7

Entry: 20

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem is that each thread needs a lock on the vector before accessing. When multiple threads want to access the vector, they must wait for the lock to be released.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for a long time, preventing other threads from accessing the vector.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

5.0

Score 2:

5

Score 3:

5

Entry: 21

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to others?

Answer:

The problem of this approach is, when parallelism is enabled, this approach is slower than others. This is because the other approaches use a global lock to protect the acquisition of all the other locks.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is too coarse.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15

Score 2:

15

Score 3:

15

Score outlier:

5

Entry: 22

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to global-order?

Answer:

Without -p, it is worse than global-order but better than try-wait. It gets 0.10 seconds in -n 2 -l 100000 while global-order gets 0.05 seconds.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

12

Score 3:

10

Entry: 23

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

the problem for this approach is that the global lock will be under contention at all times. Resulting in its performance being the lowest of the three.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for the entire duration of the program.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

5

Score 3:

7

Entry: 24

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its perform

Answer:

The main problem is that the global lock holds the lock even when sometimes it is not necessary to avoid deadlock, it pla

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

13.0

Score 2:

10

Score 3:

10

Score outlier:

3

Entry: 25

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two?

Answer:

t-l 100000 -d Indicates the number of variable threads 2 threads :~0.7 seconds 4 threads :~1.4 seconds 8 threads :2.9-3.0 seconds

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

11

Score 2:

10

Score 3:

10

Entry: 26

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its perform

Answer:

It presents a challenge as it necessitates knowing in advance which locks to acquire, resulting in reduced concurrency. It

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

5.0

Score 2:

9

Score 3:

7

Entry: 27

Item: 3

Question:

The main problem with the vector-avoid-hold-and-wait.c approach is the potential for deadlocks. It tries to acquire locks for

Answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain “main problem” (5 points)
- For explain the setting “without the -p flag” (5 points)
- For explain the setting “with the -p flag” (5 points)

Full points:

15

Score 1:

15

Score 2:

13

Score 3:

15

Entry: 28

Item: 3

Question:

What is the main problem with the approach in vector-avoid-hold-and-wait.c? How does its performance compare to vector

Answer:

The main problem is that it requires knowing exactly which lock to acquire in advance and decreases concurrency. When

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain “main problem” (5 points)
- For explain the setting “without the -p flag” (5 points)
- For explain the setting “with the -p flag” (5 points)

Full points:

15

Score 1:

8

Score 2:

7

Score 3:

5

Entry: 29

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other approaches?

Answer:

The main problem with the vector-avoid-hold-and-wait.c approach is the use of a global lock, which introduces a bottleneck.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

8

Score 2:

10

Score 3:

9

Score outlier:

15

Entry: 30

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two?

Answer:

The main problem with this approach is too coarse, since the global lock will be under contention even when the vectors are not being updated.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other vectors) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10

Score 2:

12

Score 3:

10

Entry: 31

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem is it uses global lock. It will reduce concurrency. May have lower performance with -p. Without -p, performance is better.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

5

Score 2:

7

Score 3:

5

Entry: 32

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem of this approach is that it requires us to know exactly which locks to acquire in prior and decreases the number of locks that are held at any one time.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for a long time, and this can cause a significant performance penalty.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

13.0

Score 2:

15

Score 3:

13

Entry: 33

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem is that the global lock will be under contention. When running without -p, the performance is slower compared to the other two approaches.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

7

Score 3:

7

Entry: 34

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The problem with this approach is that it uses a single lock around acquisition, and this lock will be acquired most of the time by the first thread to acquire it.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other threads) is held by the first thread to acquire it.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

12

Score 3:

10

Entry: 35

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other approaches?

Answer:

With this approach, all threads are blocked if one thread is waiting for a mutex even if other threads can acquire all the other mutexes.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other mutexes) is held for too long.

Sample criteria:

three subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15

Score 2:

10

Score 3:

13

Entry: 36

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two?

Answer:

The presence of a global lock for all vectors implies that even when different threads are attempting to manipulate them, all threads must wait for the lock to be released.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other vectors) is held by one thread at a time, even when multiple threads are attempting to acquire it.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15

Score 2:

15

Score 3:

15

Entry: 37

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its perform

Answer:

The global lock which protects the acquisition of all other locks will be subject to contention even if the vector of each thre

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

10

Score 3:

10

Entry: 38

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other two approaches?

Answer:

The main problem is too coarse: the global lock (which protects the acquisition of all the other locks) will be under contention for a long time.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) will be under contention for a long time.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

15.0

Score 2:

13

Score 3:

15

Entry: 39

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other approaches?

Answer:

It employs a universal lock across all vectors, indicating that even when various threads are engaged, a thread may struggle to acquire the lock.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held by a thread that is not the one that needs it.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

10.0

Score 2:

15

Score 3:

15

Score outlier:

5

Entry: 40

Item: 3

Question:

Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other approaches?

Answer:

Using -p will decrease the running time.

Sample answer:

The main problem with this approach is that it is too coarse: the global lock (which protects the acquisition of all the other locks) is held for too long.

Sample criteria:

three-subquestions, 5 points for each subquestion.

- For explain "main problem" (5 points)
- For explain the setting "without the -p flag" (5 points)
- For explain the setting "with the -p flag" (5 points)

Full points:

15

Score 1:

1.0

Score 2:

0

Score 3:

0

Score outlier:

5